

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.wireadminRe-`
lease-1.0.3-SNAPSHOT

September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
108	Wire Admin Service Specification	7
108.1	Introduction	7
108.2	Producer Service	10
108.3	Consumer Service	12
108.4	Implementation issues	14
108.5	Wire Properties	14
108.6	Composite objects	16
108.7	Wire Flow Control	19
108.8	Flavors	22
108.9	Converters	22
108.10	Wire Admin Service Implementation	23
108.11	Wire Admin Listener Service Events	23
108.12	Connecting External Entities	25
108.13	Related Standards	26
108.14	Security	26
108.15	org.osgi.service.wireadmin	27
108.16	References	43

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

108 Wire Admin Service Specification

Version 1.0

108.1 Introduction

The Wire Admin service is an administrative service that is used to control a wiring topology in the OSGi Framework. It is intended to be used by user interfaces or management programs that control the wiring of services in an OSGi Framework.

The Wire Admin service plays a crucial role in minimizing the amount of context-specific knowledge required by bundles when used in a large array of configurations. The Wire Admin service fulfills this role by dynamically *wiring* services together. Bundles participate in this wiring process by registering services that produce or consume data. The Wire Admin service *wires* the services that produce data to services which consume data.

The purpose of wiring services together is to allow configurable cooperation of bundles in an OSGi Framework. For example, a temperature sensor can be connected to a heating module to provide a controlled system.

The Wire Admin service is a very important OSGi configuration service and is designed to cooperate closely with the Configuration Admin service, as defined in ???.

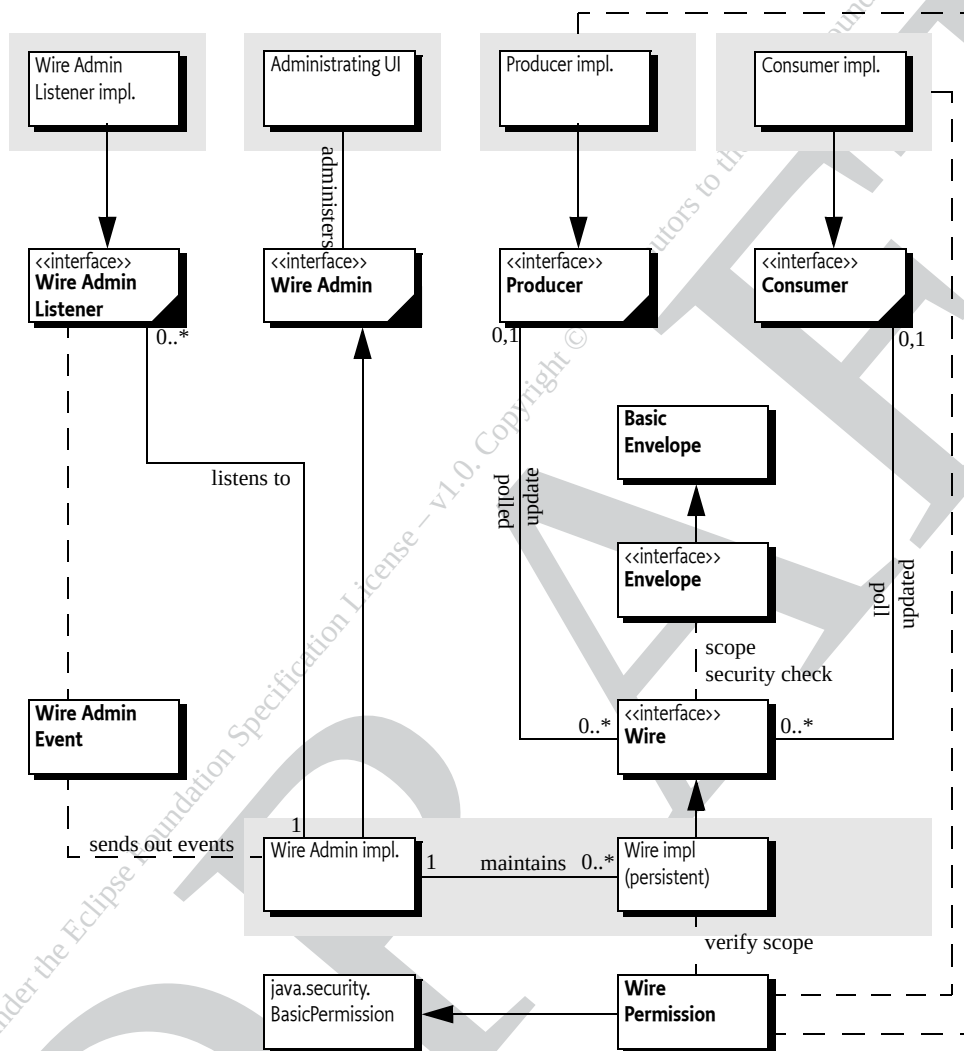
108.1.1 Wire Admin Service Essentials

- *Topology Management* - Provide a comprehensive mechanism to link data-producing components with data-consuming components in an OSGi environment.
- *Configuration Management* - Contains configuration data in order to allow either party to adapt to the special needs of the wire.
- *Data Type Handling* - Facilitate the negotiation of the data type to be used for data transfer between producers of data and consumers of data. Consumers and producers must be able to handle multiple data types for data exchanges using a preferred order.
- *Composites* - Support producers and consumers that can handle a large number of data items.
- *Security* - Separate connected parties from each other. Each party must not be required to hold the service object of the other party.
- *Simplicity* - The interfaces should be designed so that both parties, the Producer and the Consumer services, should be easy to implement.

108.1.2 Wire Admin Service Entities

- *Producer* - A service object that generates information to be used by a Consumer service.
- *Consumer* - A service object that receives information generated by a Producer service.
- *Wire* - An object created by the Wire Admin service that defines an association between a Producer service and a Consumer service. Multiple Wire objects can exist between the same Producer and Consumer pair.
- *WireAdmin* - The service that provides methods to create, update, remove, and list Wire objects.
- *WireAdminListener* - A service that receives events from the Wire Admin service when the Wire object is manipulated or used.

- *WireAdminEvent* - The event that is sent to a *WireAdminListener* object, describing the details of what happened.
- *Configuration Properties* - Properties that are associated with a *Wire* object and that contain identity and configuration information set by the administrator of the Wire Admin service.
- *PID* - The Persistent IDentity as defined in the Configuration Admin specification.
- *Flavors* - The different data types that can be used to exchange information between Producer and Consumer services.
- *Composite Producer/Consumer* - A Producer/Consumer service that can generate/accept different kinds of values.
- *Envelope* - An interface for objects that can identify a value that is transferred over the wire. Envelope objects contain also a scope name that is used to verify access permissions.
- *Scope* - A set of names that categorizes the kind of values contained in Envelope objects for security and selection purposes.
- *Basic Envelope* - A concrete implementation of the Envelope interface.
- *WirePermission* - A Permission sub-class that is used to verify if a Consumer service or Producer service has permission for specific scope names.
- *Composite Identity* - A name that is agreed between a composite Consumer and Producer service to identify the kind of objects that they can exchange.

Figure 108.1 Class Diagram, *org.osgi.service.wireadmin*

108.1.3

Operation Summary

The Wire Admin service maintains a set of persistent Wire objects. A Wire object contains a Persistent Identity (PID) for a Consumer service and a PID for a Producer service. (Wire objects can therefore be created when the Producer or Consumer service is not registered.)

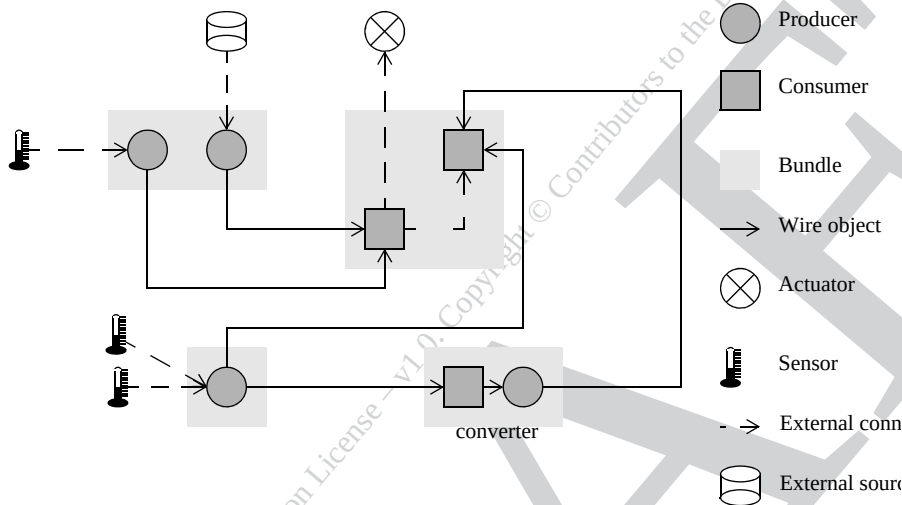
If both those Producer and Consumer services are registered with the Framework, they are connected by the Wire Admin service. The Wire Admin service calls a method on each service object and provides the list of Wire objects to which they are connected.

When a Producer service has new information, it should send this information to each of the connected Wire objects. Each Wire object then must check the filtering and security. If both filtering and security allow the transfer, the Producer service should inform the associated Consumer service with the new information. The Consumer services can also poll a Wire object for a new value at any time.

When a Consumer or Producer service is unregistered from the OSGi Framework, the other object in the association is informed that the Wire object is no longer valid.

Administrative applications can use the Wire Admin service to create and delete wires. These changes are immediately reflected in the current topology and are broadcast to Wire Admin Listener services.

Figure 108.2 An Example Wiring Scheme in an OSGi Environment



108.2 Producer Service

A Producer is a service that can produce a sequence of data objects. For example, a Producer service can produce, among others, the following type of objects:

- Measurement objects that represent a sensor measurement such as temperature, movement, or humidity.
- A String object containing information for user consumption, such as headlines.
- A Date object indicating the occurrence of a periodic event.
- Position information.
- Envelope objects containing status items which can be any type.

108.2.1 Producer Properties

A Producer service must be registered with the OSGi Framework under the interface name `org.osgi.service.wireadmin.Producer`. The following service properties must be set:

- `service.pid` - The value of this property, also known as the PID, defines the Persistent IDentity of a service. A Producer service must always use the same PID value whenever it is registered. The PID value allows the Wire Admin service to consistently identify the Producer service and create a persistent Wire object that links a Producer service to a Consumer service. See [1] *Design Patterns* specification for the rules regarding PIDs.
- `wireadmin.producer.flavors` - The value of this property is an array of Class objects (`Class[]`) that are the classes of the objects the service can produce. See *Flavors* on page 22 for more information about the data type negotiation between Producer and Consumer services.
- `wireadmin.producer.filters` - This property indicates to the Wire Admin service that this Producer service performs its own update filtering, meaning that the consumer can limit the number of update calls with a filter expression. This does not modify the data; it only determines whether an update via the wire occurs. If this property is not set, the Wire object must filter according to

the description in *Composite objects* on page 16 . This service registration property does not need to have a specific value.

- `wireadmin.producer.scope` - Only for a composite Producer service, a list of scope names that define the scope of this Producer service, as explained in *Scope* on page 17 .
- `wireadmin.producer.composite` - List the composite identities of Consumer services with which this Producer service can interoperate. This property is of type `String[]`. A composite Consumer service can inter-operate with a composite Producer service when there is at least one name that occurs in both the Consumer service's array and the Producer service's array for this property.

108.2.2

Connections

The Wire Admin service connects a Producer service and a Consumer service by creating a Wire object. If the Consumer and Producer services that are bound to a Wire object are registered with the Framework, the Wire Admin service must call the `consumersConnected(Wire[])` method on the Producer service object. Every change in the Wire Admin service that affects the Wire object to which a Producer service is connected must result in a call to this method. This requirement ensures that the Producer object is informed of its role in the wiring topology. If the Producer service has no Wire objects attached when it is registered, the Wire Admin service must always call `consumersConnected(null)`. This situation implies that a Producer service can assume it always gets called back from the Wire Admin service when it registers.

108.2.3

Producer Example

The following example shows a clock producer service that sends out a Date object every second.

```
public class Clock extends Thread implements Producer {
    Wire        wires[];
    BundleContext context;
    boolean      quit;

    Clock( BundleContext context ) {
        this.context = context;
        start();
    }

    public synchronized void run() {
        Hashtable p = new Hashtable();
        p.put( org.osgi.service.wireadmin.WireConstants.
            WIREADMIN_PRODUCER_FLAVORS,
            new Class[] { Date.class } );
        p.put( org.osgi.framework.Constants.SERVICE_PID,
            "com.acme.clock" );
        context.registerService(
            Producer.class.getName(), this, p );

        while( ! quit )
            try {
                Date now = new Date();
                for( int i=0; wires!=null && i<wires.length;i++ )
                    wires[i].update( now );
                wait( 1000 );
            }
            catch( InterruptedException ie) {
                /* will recheck quit */
            }
    }

    public void synchronized consumersConnected(Wire wires[])
```

```

    {
        this.wires = wires;
    }
    public Object polled(Wire wire) { return new Date(); }
    ...
}

```

108.2.4 Push and Pull

Communication between Consumer and Producer services can be initiated in one of the following ways.

- The Producer service calls the `update(Object)` method on the Wire object. The Wire object implementation must then call the `updated(Wire, Object)` method on the Consumer service, if the filtering allows this.
- The Consumer service can call `poll()` on the Wire object. The Wire object must then call `polled(Wire)` on the Producer object. Update filtering must not apply to polling.

108.2.5 Producers and Flavors

Consumer services can only understand specific data types, and are therefore restricted in what data they can process. The acceptable object classes, the flavors, are communicated by the Consumer service to the Wire Admin service using the Consumer service's service registration properties. The method `getFlavors()` on the Wire object returns this list of classes. This list is an ordered list in which the first class is the data type that is the most preferred data type supported by the Consumer service. The last class is the least preferred data type. The Producer service must attempt to convert its data into one of the data types according to the preferred order, or will return null from the poll method to the Consumer service if none of the types are recognized.

Classes cannot be easily compared for equivalence. Sub-classes and interfaces allow classes to masquerade as other classes. The `Class.isAssignableFrom(Class)` method verifies whether a class is type compatible, as in the following example:

```

Object polled(Wire wire) {
    Class clazzes[] = wire.getFlavors();
    for ( int i=0; i<clazzes.length; i++ ) {
        Class clazz = clazzes[i];
        if ( clazz.isAssignableFrom( Date.class ) )
            return new Date();
        if ( clazz.isAssignableFrom( String.class ) )
            return new Date().toString();
    }
    return null;
}

```

The order of the if statements defines the preferences of the Producer object. Preferred data types are checked first. This order normally works as expected but in rare cases, sub-classes can change it. Normally, however, that is not a problem.

108.3 Consumer Service

A Consumer service is a service that receives information from one or more Producer services and is wired to Producer services by the Wire Admin service. Typical Consumer services are as follows:

- The control of an actuator, such as a heating element, oven, or electric shades
- A display

- A log
- A state controller such as an alarm system

108.3.1 Consumer Properties

A Consumer service must be registered with the OSGi Framework under the interface name `org.osgi.service.wireadmin.Consumer`. The following service properties must be set:

- `service.pid` - The value of this property, also known as the PID, defines the Persistent IDentity of a service. A Consumer service must always use the same PID value whenever it is registered. The PID value allows the Wire Admin service to consistently identify the Consumer service and create a persistent Wire object that links a Producer service to a Consumer service. See the Configuration Admin specification for the rules regarding PIDs.
- `wireadmin.consumer.flavors` - The value of this property is an array of Class objects (`Class[]`) that are the acceptable classes of the objects the service can process. See *Flavors* on page 22 for more information about the data type negotiation between Producer and Consumer services.
- `wireadmin.consumer.scope` - Only for a composite Consumer service, a list of scope names that define the scope of this Consumer service, as explained in *Scope* on page 17 .
- `wireadmin.consumer.composite` - List the composite identities of Producer services that this Consumer service can interoperate with. This property is of type `String[]`. A composite Consumer service can interoperate with a composite Producer service when at least one name occurs in both the Consumer service's array and the Producer service's array for this property.

108.3.2 Connections

When a Consumer service is registered and a Wire object exists that associates it to a registered Producer service, the `producersConnected(Wire[])` method is called on the Consumer service.

Every change in the Wire Admin service that affects a Wire object to which a Consumer service is connected must result in a call to the `producersConnected(Wire[])` method. This rule ensures that the Consumer object is informed of its role in the wiring topology. If the Consumer service has no Wire objects attached, the argument to the `producersConnected(Wire[])` method must be null. This method must also be called when a Producer service registers for the first time and no Wire objects are available.

108.3.3 Consumer Example

For example, a service can implement a Consumer service that logs all objects that are sent to it in order to allow debugging of a wiring topology.

```
public class LogConsumer implements Consumer{
    public LogConsumer( BundleContext context ) {
        Hashtable ht = new Hashtable();
        ht.put(
            Constants.SERVICE_PID, "com.acme.logconsumer" );
        ht.put( WireConstants.WIREADMIN_CONSUMER_FLAVORS,
            new Class[] { Object.class } );
        context.registerService( Consumer.class.getName(),
            this, ht );
    }
    public void updated( Wire wire, Object o ) {
        getLog().log( LogService.LOG_INFO, o.toString() );
    }
    public void producersConnected( Wire [] wires) {}
    LogService getLog() { ... }
}
```

108.3.4 Polling or Receiving a Value

When the Producer service produces a new value, it calls the `update(Object)` method on the Wire object, which in turn calls the `updated(Wire, Object)` method on the Consumer service object. When the Consumer service needs a value immediately, it can call the `poll()` method on the Wire object which in turn calls the `polled(Wire)` method on the Producer service.

If the `poll()` method on the Wire object is called and the Producer is unregistered, it must return a null value.

108.3.5 Consumers and Flavors

Producer objects send objects of different data types through Wire objects. A Consumer service object should offer a list of preferred data types (classes) in its service registration properties. The Producer service, however, can still send a null object or an object that is not of the preferred types. Therefore, the Consumer service must check the data type and take the appropriate action. If an object type is incompatible, then a log message should be logged to allow the operator to correct the situation.

The following example illustrates how a Consumer service can handle objects of type Date, Measurement, and String.

```
void process( Object in ) {
    if ( in instanceof Date )
        processDate( (Date) in );
    else if ( in instanceof Measurement )
        processMeasurement( (Measurement) in );
    else if ( in instanceof String )
        processString( (String) in );
    else
        processError( in );
}
```

108.4 Implementation issues

The Wire Admin service can call the `consumersConnected` or `producersConnected` methods during the registration of the Consumer or Producer service. Care should be taken in this method call so that no variables are used that are not yet set, such as the `ServiceRegistration` object that is returned from the registration. The same is true for the `updated` or `polled` callback because setting the Wire objects on the Producer service causes such a callback from the `consumersConnected` or `producersConnected` method.

A Wire Admin service must call the `producersConnected` and `consumersConnected` method asynchronously from the registrations, meaning that the Consumer or Producer service can use synchronized to restrict access to critical variables.

When the Wire Admin service is stopped, it must disconnect all connected consumers and producers by calling `producersConnected` and `consumersConnected` with a null for the wires parameter.

108.5 Wire Properties

A Wire object has a set of properties (a Dictionary object) that configure the association between a Consumer service and a Producer service. The type and usage of the keys, as well as the allowed types for the values are defined in ???.

The Wire properties are explained in the following table.

Table 108.1 Standard Wire Properties

Constant	Description
WIREADMIN_PID	The value of this property is a unique Persistent IDentity as defined in chapter ???. This PID must be automatically created by the Wire Admin service for each new Wire object.
WIREADMIN_PRODUCER_PID	The value of the property is the PID of the Producer service.
WIREADMIN_CONSUMER_PID	The value of this property is the PID of the Consumer service.
WIREADMIN_FILTER	The value of this property is an OSGi filter string that is used to control the update of produced values. This filter can contain a number of attributes as explained in <i>Wire Flow Control</i> on page 19 .

The properties associated with a Wire object are not limited to the ones defined in Table 108.1 . The Dictionary object can also be used for configuring *both* Consumer services and Producer services. Both services receive the Wire object and can inspect the properties and adapt their behavior accordingly.

108.5.1 Display Service Example

In the following example, the properties of a Wire object, which are set by the Operator or User, are used to configure a Producer service that monitors a user's email account regularly and sends a message when the user has received email. This WireMail service is illustrated as follows:

```
public class WireMail extends Thread
    implements Producer {
    Wire        wires[];
    BundleContext context;
    boolean     quit;

    public void start( BundleContext context ) {
        Hashtable ht = new Hashtable();
        ht.put( Constants.SERVICE_PID, "com.acme.wiremail" );
        ht.put( WireConstants.WIREADMIN_PRODUCER_FLAVORS,
            new Class[] { Integer.class } );
        context.registerService( this,
            Producer.class.getName(),
            ht );
    }
    public synchronized void consumersConnected(
        Wire wires[] ) {
        this.wires = wires;
    }
    public Object polled( Wire wire ) {
        Dictionary p = wire.getProperties();
        // The password should be
        // obtained from User Admin Service
        int n = getNrMails(
            p.get( "userid" ),
            p.get( "mailhost" ) );
        return new Integer( n );
    }
}
```

```

    }
    public synchronized void run() {
        while ( !quit )
            try {
                for ( int i=0; wires != null && i<wires.length;i++)
                    wires[i].update( polled( wires[i] ) );

                wait( 150000 );
            }
            catch( InterruptedException e ) { break; }
        }
        ...
    }
}

```

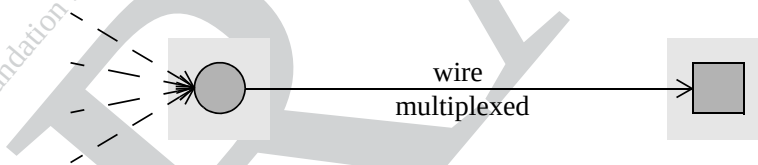
108.6 Composite objects

A Producer and/or Consumer service for each information item is usually the best solution. This solution is not feasible, however, when there are hundreds or thousands of information items. Each registered Consumer or Producer service carries the overhead of the registration, which may overwhelm a Framework implementation on smaller platforms.

When the size of the platform is an issue, a Producer and a Consumer service should abstract a larger number of information items. These Consumer and Producer services are called *composite*.

Figure 108.3

Composite Producer Example



Composite Producer and Consumer services should register respectively the [WIREADMIN_PRODUCER_COMPOSITE](#) and [WIREADMIN_CONSUMER_COMPOSITE](#) *composite identity* property with their service registration. These properties should contain a list of composite identities. These identities are not defined here, but are up to a mutual agreement between the Consumer and Producer service. For example, a composite identity could be MOST-1.5 or GSM-Phase2-Terminal. The name may follow any scheme but will usually have some version information embedded. The composite identity properties are used to match Consumer and Producer services with each other during configuration of the Wire Admin service. A Consumer and Producer service should inter-operate when at least one equal composite identity is listed in both the Producer and Consumer composite identity service property.

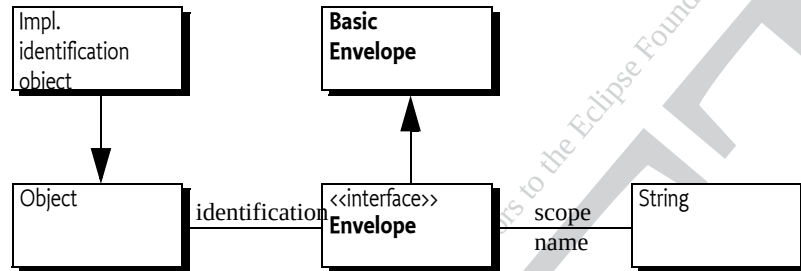
Composite producers/consumers must identify the *kind* of objects that are transferred over the Wire object, where *kind* refers to the intent of the object, not the data type. For example, a Producer service can represent the status of a door-lock and the status of a window as a boolean. If the status of the window is transferred as a boolean to the Consumer service, how would it know that this boolean represents the window and not the door-lock

To avoid this confusion, the Wire Admin service includes an Envelope interface. The purpose of the Envelope interface is to associate a value object with:

- An identification object
- A scope name

Figure 108.4

Envelope



108.6.1 Identification

The Envelope object's identification object is used to identify the value carried in the Envelope object. Each unique kind of value must have its own unique identification object. For example, a left-front-window should have a different identification object than a rear-window.

The identification is of type Object. Using the Object class allows String objects to be used, but also makes it possible to use more complex objects. These objects can convey information in a way that is mutually agreed between the Producer and Consumer service. For example, its type may differ depending on each kind of value so that the *Visitor* pattern, see [1] *Design Patterns*, can be used. Or it may contain specific information that makes the Envelope object easier to dispatch for the Consumer service.

108.6.2 Scope

The scope name is a String object that *categorizes* the Envelope object. The scope name is used to limit the kind of objects that can be exchanged between composite Producer and Consumer services, depending on security settings.

The name-space for this scope should be mutually agreed between the Consumer and Producer services a priori. For the Wire Admin service, the scope name is an opaque string. Its syntax is specified in *Scope name syntax* on page 19.

Both composite Producer and Consumer services must add a list of their supported scope names to the service registration properties. This list is called the *scope* of that service. A Consumer service must add this scope property with the name of `WIREADMIN_CONSUMER_SCOPE`, a Producer service must add this scope property with the name `WIREADMIN_PRODUCER_SCOPE`. The type of this property must be a `String[]` object.

Not registering this property by the Consumer or the Producer service indicates to the Wire Admin service that any Wire object connected to that service must return null for the Wire. `getScope()` method. This case must be interpreted by the Consumer or Producer service that no scope verification is taking place. Secure Producer services should not produce values for this Wire object and secure Consumer services should not accept values.

It is also allowed to register with a *wildcard*, indicating that all scope names are supported. In that case, the `WIREADMIN_SCOPE_ALL` (which is `String[] { "*" }`) should be registered as the scope of the service. The Wire object's scope is then fully defined by the other service connected to the Wire object.

The following example shows how a scope is registered.

```
static String [] scope = { "DoorLock", "DoorOpen", "VIN" };

public void start( BundleContext context ) {
    Dictionary properties = new Hashtable();
    properties.put(
        WireConstants.WIREADMIN_CONSUMER_SCOPE,
```

```
        scope );
properties.put( WireConstants.WIREADMIN_CONSUMER_PID,
    "com.acme.composite.consumer" );
properties.put(
    WireConstants.WIREADMIN_CONSUMER_COMPOSITE,
    new String[] { "OSGiSP-R3" } );
context.registerService( Consumer.class.getName(),
    new AcmeConsumer(),
    properties );
}
```

Both a composite Consumer and Producer service must register a scope to receive scope support from the Wire object. These two scopes must be converted into a single Wire object's scope and scope names in this list must be checked for the appropriate permissions. This resulting scope is available from the Wire. `getScope()` method.

If no scope is set by either the Producer or the Consumer service the result must be null. In that case, the Producer or Consumer service must assume that no security checking is in place. A secure Consumer or Producer service should then refuse to operate with that Wire object.

Otherwise, the resulting scope is the intersection of the Consumer and Producer service scope where each name in the scope, called *m*, must be implied by a `WirePermission[m,CONSUME]` of the Consumer service, and `WirePermission[m,PRODUCE]` of the Producer service.

If either the Producer or Consumer service has registered a wildcard scope then it must not restrict the list of the other service, except for the permission check. If both the Producer and Consumer service registered a wild-card, the resulting list must be `WIREADMIN_SCOPE_ALL` (`String[]{"*"}`).

For example, the Consumer service has registered a scope of {A,B,C} and has `WirePermission[*,CONSUME]`. The Producer service has registered a scope of {B,C,E} and has `WirePermission[C|E, PRODUCE,]`. The resulting scope is then {C}. The following table shows this and more examples.

Table 108.2 Examples of scope calculation. C=Consumer, P=Producer, p=WirePermission, s=scope

Cs	Cp	Ps	Pp	Wire Scope
null		null		null
{A,B,C}	*	null		null
null		{C,D,E}		null
{A,B,C}	B C	{A,B,C}	A B	{B}
*	*	{A,B,C}	A B C	{A,B,C}
*	*	*	*	{*}
{A,B,C}	A B C	{A,B,C}	X	{}
{A,B,C}	*	{B,C,E}	C E	{C}

The Wire object's scope must be calculated only once, when both the Producer and Consumer service become connected. When a Producer or Consumer service subsequently modifies its scope, the Wire object must *not* modify the original scope. A Consumer and a Produce service can thus assume that the scope does not change after the `producersConnected` method or `consumersConnected` method has been called.

108.6.3

Access Control

When an `Envelope` object is used as argument in `Wire.update(Object)` then the Wire object must verify that the `Envelope` object's scope name is included in the Wire object's scope. If this is not the case, the update must be ignored (the updated method on the Consumer service must not be called).

A composite Producer represents a number of values, which is different from a normal Producer that can always return a single object from the poll method. A composite Producer must therefore return an array of Envelope objects (Envelope[]). This array must contain Envelope objects for all the values that are in the Wire object's scope. It is permitted to return all possible values for the Producer because the Wire object must remove all Envelope objects that have a scope name not listed in the Wire object's scope.

108.6.4 Composites and Flavors

Composite Producer and Consumer services must always use a flavor of the Envelope class. The data types of the values must be associated with the scope name or identification and mutually agreed between the Consumer and Producer services.

Flavors and Envelope objects both represent categories of different values. Flavors, however, are different Java classes that represent the same kind of value. For example, the tire pressure of the left front wheel could be passed as a Float, an Integer, or a Measurement object. Whatever data type is chosen, it is still the tire pressure of the left front wheel. The Envelope object represents the kind of object, for example the right front wheel tire pressure, or the left rear wheel.

108.6.5 Scope name syntax

Scope names are normal String objects and can, in principle, contain any Unicode character. In use, scope names can be a full wildcard (*) but they cannot be partially wildcarded for matching scopes.

Scope names are used with the WirePermission class that extends java.security.BasicPermission. The BasicPermission class implements the implies method and performs the name matching. The wildcard matching of this class is based on the concept of names where the constituents of the name are separated with a period ('): for example, org.osgi.service.http.port.

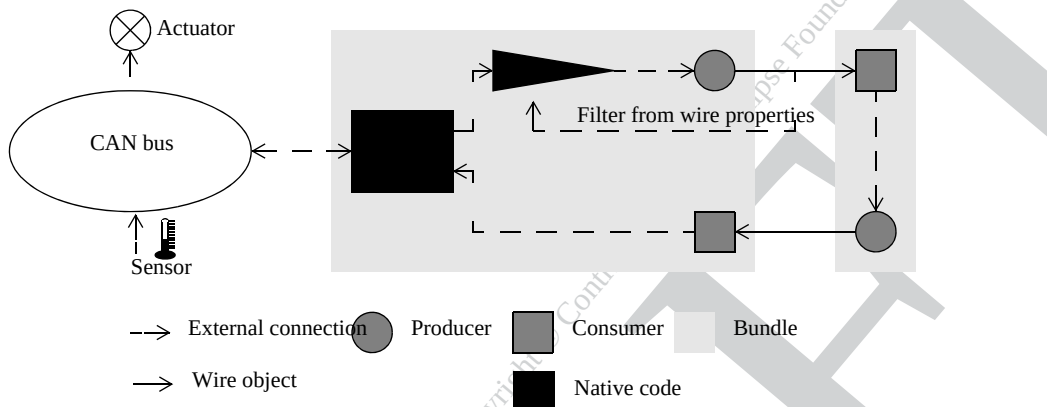
Scope names must therefore follow the rules for fully qualified Java class names. For example, door.lock is a correct scope name while door-lock is not.

108.7 Wire Flow Control

The WIREADMIN_FILTER property contains a filter expression (as defined in the OSGi Framework Filter class) that is used to limit the number of updates to the Consumer service. This is necessary because information can arrive at a much greater rate than can be processed by a Consumer service. For example, a single CAN bus (the electronic control bus used in current cars) in a car can easily deliver hundreds of measurements per second to an OSGi based controller. Most of these measurements are not relevant to the OSGi bundles, at least not all the time. For example, a bundle that maintains an indicator for the presence of frost is only interested in measurements when the outside temperature passes the 4 degrees Celsius mark.

Limiting the number of updates from a Producer service can make a significant difference in performance (meaning that less hardware is needed). For example, a vendor can implement the filter in native code and remove unnecessary updates prior to processing in the Java Virtual Machine (JVM). This is depicted in Figure 108.5 on page 20 .

Figure 108.5 Filtering of Updates



The filter can use any combination of the following attributes in a filter to implement many common filtering schemes:

Table 108.3 Filter Attribute Names

Constant	Description
WIREVALUE_CURRENT	Current value of the data from the Producer service.
WIREVALUE_PREVIOUS	Previous data value that was reported to the Consumer service.
WIREVALUE_DELTA_ABSOLUTE	The actual positive difference between the previous data value and the current data value. For example, if the previous data value was 3 and the current data value is -0.5, then the absolute delta is 3.5. This filter attribute is not set when the current or previous value is not a number.
WIREVALUE_DELTA_RELATIVE	The absolute (meaning always positive) relative change between the current and the previous data values, calculated with the following formula: $ \text{previous} - \text{current} / \text{current} $. For example, if the previous value was 3 and the new value is 5, then the relative delta is $ 3 - 5 / 5 = 0.4$. This filter attribute is not set when the current or previous value is not a number.
WIREVALUE_ELAPSED	The time in milliseconds between the last time the Consumer. <code>updated(Wire, Object)</code> returned and the time the filter is evaluated.

Filter attributes can be used to implement many common filtering schemes that limit the number of updates that are sent to a Consumer service. The Wire Admin service specification requires that updates to a Consumer service are always filtered if the `WIREADMIN_FILTER` Wire property is present. Producer services that wish to perform the filtering themselves should register with a service property `WIREADMIN_PRODUCER_FILTERS`. Filtering must be performed by the Wire object for all other Producer services.

Filtering for composite Producer services is not supported. When a filter is set on a Wire object, the Wire must still perform the filtering (which is limited to time filtering because an Envelope object is not a magnitude), but this approach may lose relevant information because the objects are of a different kind. For example, an update of every 500 ms could miss all speed updates because there is a wheel pressure update that resets the elapsed time. Producer services should, however, still implement a filtering scheme that could use proprietary attributes to filter on different kind of objects.

108.7.1 Filtering by Time

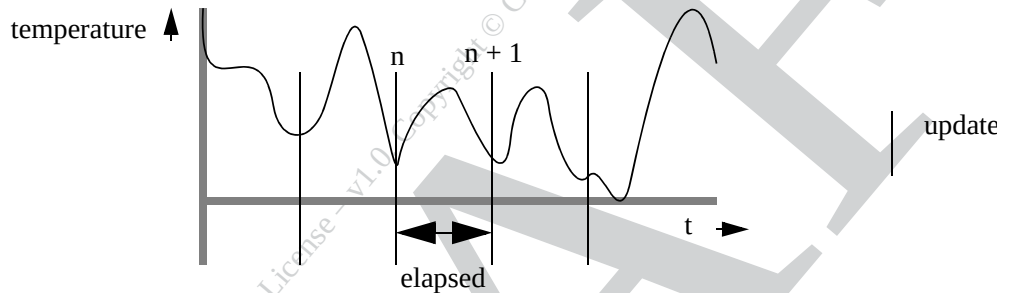
The simplest filter mechanism is based on time. The `wirevalue.elapsed` attribute contains the amount of milliseconds that have passed since the last update to the associated Consumer service. The following example filter expression illustrates how the updates can be limited to approximately 40 times per minute (once every 1500 ms).

```
(wirevalue.elapsed>=1500)
```

Figure 108.6 depicts this example graphically.

Figure 108.6

Elapsed Time Change



108.7.2 Filtering by Change

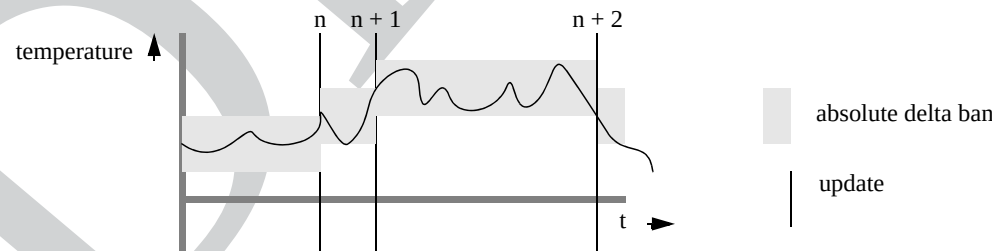
A Consumer service is often not interested in an update if the data value has not changed. The following filter expression shows how a Consumer service can limit the updates from a temperature sensor to be sent only when the temperature has changed at least 1 °K.

```
(wirevalue.delta.absolute>=1)
```

Figure 108.7 depicts a band that is created by the absolute delta between the previous data value and the current data value. The Consumer is only notified with the `updated(Wire, Object)` method when a data value is outside of this band.

Figure 108.7

Absolute Delta

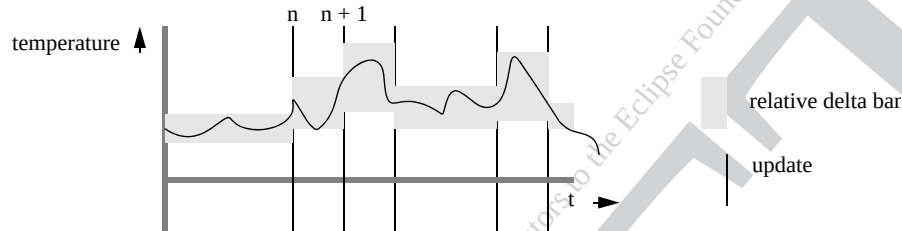


The delta may also be relative. For example, if a car is moving slowly, then updates for the speed of the car are interesting even for small variations. When a car is moving at a high rate of speed, updates are only interesting for larger variations in speed. The following example shows how the updates can be limited to data value changes of at least 10%.

```
(wirevalue.delta.relative>=0.1)
```

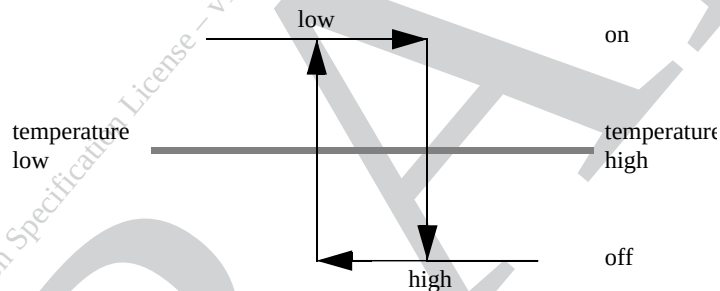
Figure 108.8 on page 22 depicts a relative band. Notice that the size of the band is directly proportional to the size of the sample value.

Figure 108.8

Relative Delta (not to scale)**108.7.3****Hysteresis**

A thermostat is a control device that usually has a hysteresis, which means that a heater should be switched on below a certain specified low temperature and should be switched off at a specified high temperature, where *high* > *low*. This is graphically depicted in Figure 108.9 on page 22 . The specified acceptable temperatures reduce the amount of start/stops of the heater.

Figure 108.9

Hysteresis

A Consumer service that controls the heater is only interested in events at the top and bottom of the hysteresis. If the specified high value is 250 °K and the specified low value is 249 °K, the following filter illustrates this concept:

```
(| (&(wirevalue.previous<=250) (wirevalue.current>250))
  (&(wirevalue.previous>=249) (wirevalue.current<249))
)
```

108.8**Flavors**

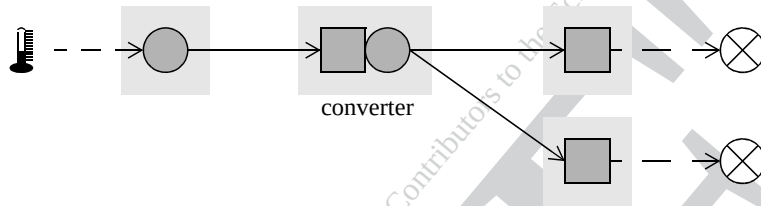
Both Consumer and Producer services should register with a property describing the classes of the data types they can consume or produce respectively. The classes are the *flavors* that the service supports. The purpose of flavors is to allow an administrative user interface bundle to connect Consumer and Producer services. Bundles should only create a connection when there is at least one class shared between the flavors from a Consumer service and a Producer service. Producer services are responsible for selecting the preferred object type from the list of the object types preferred by the Consumer service. If the Producer service cannot convert its data to any of the flavors listed by the Consumer service, null should be used instead.

108.9**Converters**

A converter is a bundle that registers a Consumer and a Producer service that are related and performs data conversions. Data values delivered to the Consumer service are processed and transferred

via the related Producer service. The Producer service sends the converted data to other Consumer services. This is shown in Figure 108.10.

Figure 108.10 Converter (for legend see Figure 108.2)



108.10 Wire Admin Service Implementation

The Wire Admin service is the administrative service that is used to control the wiring topology in the OSGi Framework. It contains methods to create or update wires, delete wires, and list existing wires. It is intended to be used by user interfaces or management programs that control the wiring topology of the OSGi Framework.

The `createWire(String,String,Dictionary)` method is used to associate a Producer service with a Consumer service. The method always creates and returns a new object. It is therefore possible to create multiple, distinct wires between a Producer and a Consumer service. The properties can be used to create multiple associations between Producer and Consumer services in that act in different ways.

The properties of a Wire object can be updated with the `update(Object)` method. This method must update the properties in the Wire object and must notify the associated Consumer and Producer services if they are registered. Wire objects that are no longer needed can be removed with the `deleteWire(Wire)` method. All these methods are in the WireAdmin class and not in the Wire class for security reasons. See *Security* on page 26.

The `getWires(String)` method returns an array of Wire objects (or null). All objects are returned when the filter argument is null. Specifying a filter argument limits the returned objects. The filter uses the same syntax as the Framework Filter specification. This filter is applied to the properties of the Wire object and only Wire objects that match this filter are returned.

The following example shows how the `getWires` method can be used to print the PIDs of Producer services that are wired to a specific Consumer service.

```
String f = "(wireadmin.consumer.pid=com.acme.x)";
Wire [] wires = getWireAdmin().getWires( f );
for ( int i=0; wires != null && i < wires.length;i++ )
    System.out.println(
        wires[i].getProperties().get(
            "wireadmin.producer.pid"
        )
    );
```

108.11 Wire Admin Listener Service Events

The Wire Admin service has an extensive list of events that it can deliver. The events allow other bundles to track changes in the topology as they happen. For example, a graphic user interface program can use the events to show when Wire objects become connected, when these objects are deleted, and when data flows over a Wire object.

A bundle that is interested in such events must register a `WireAdminListener` service object with a special Integer property `WIREADMIN_EVENTS`("wireadmin.events"). This Integer object contains a bitmap of all the events in which this Wire Admin Listener service is interested (events have associated constants that can be ORed together). A Wire Admin service must not deliver events to the Wire Admin Listener service when that event type is not in the bitmap. If no such property is registered, no events are delivered to the Wire Admin Listener service.

The `WireAdminListener` interface has only one method: `wireAdminEvent(WireAdminEvent)`. The argument is a `WireAdminEvent` object that contains the event type and associated data.

A `WireAdminEvent` object can be sent asynchronously but must be ordered for each Wire Admin Listener service. The way events must be delivered is the same as described in *Delivering Events of ???*. Wire Admin Listener services must not assume that the state reflected by the event is still true when they receive the event.

The following types are defined for a `WireEvent` object:

Table 108.4

Event type	Description
WIRE_CREATED	A new Wire object has been created.
WIRE_CONNECTED	Both the Producer service and the Consumer service are registered but may not have executed their respective <code>connectedProducers/connectedConsumers</code> methods.
WIRE_UPDATED	The Wire object's properties have been updated.
WIRE_TRACE	The Consumer has seen a new value, either after the Producer service has called the Wire. <code>update(Object)</code> method and the value was not filtered, or the Producer service has returned from the <code>polled(Wire)</code> method.
WIRE_DISCONNECTED	The Producer service or Consumer service have become unregistered and the Wire object is no longer connected.
WIRE_DELETED	The Wire object is deleted from the repository and is no longer available from the <code>getWires</code> method.
CONSUMER_EXCEPTION	The Consumer service generated an exception and the exception is included in the event.
PRODUCER_EXCEPTION	The Producer service generated an exception in a callback and the exception is included in the event.

108.11.1

Event Admin Service Events

Wire admin events must be sent asynchronously to the Event Admin service by the Wire Admin implementation, if present. The topic of a Wire Admin Event is one of the following:

`org/osgi/service/wireadmin/WireAdminEvent/<eventtype>`

The following event types are supported:

- WIRE_CREATED
- WIRE_CONNECTED
- WIRE_UPDATED
- WIRE_TRACE
- WIRE_DISCONNECTED
- WIRE_DELETED
- PRODUCER_EXCEPTION
- CONSUMER_EXCEPTION

The properties of a wire admin event are the following.

- event - (WireAdminEvent) The WireAdminEvent object broadcast by the Wire Admin service.

If the `getWire` method returns a non null value:

- wire - (Wire) The Wire object returned by the `getWire` method.
- wire.flavors - (String[]) The names of the classes returned by the Wire `getFlavors` method.
- wire.scope - (String[]) The scope of the Wire object, as returned by its `getScope` method.
- wire.connected - (Boolean) The result of the Wire `isConnected` method.
- wire.valid - (Boolean) The result of the Wire `isValid` method.

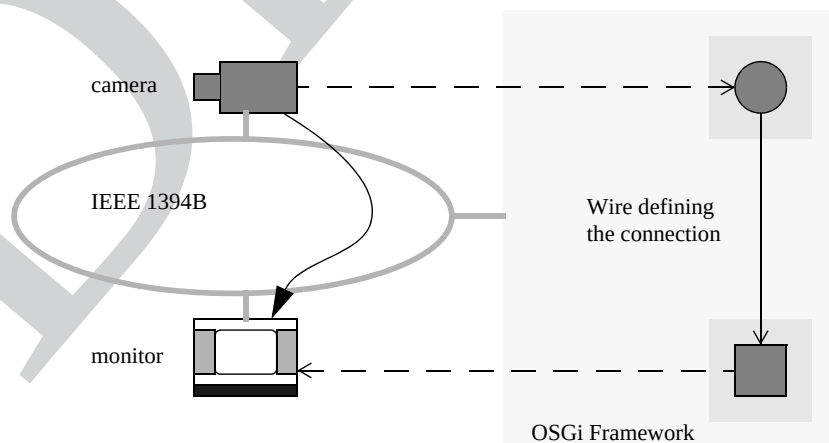
If the `getThrowable` method does not return null:

- exception - (Throwable) The Exception returned by the `getThrowable` method.
- exception.class - (String) The fully-qualified class name of the related Exception.
- exception.message - (String) The message of the related Exception
- service - (ServiceReference) The Service Reference of the Wire Admin service.
- service.id - (Long) The service id of the WireAdmin service.
- service.objectClass - (String[]) The Wire Admin service's object class (which must include `org.osgi.service.wireadmin.WireAdmin`)
- service.pid - (String) The Wire Admin service's PID.

108.12 Connecting External Entities

The Wire Admin service can be used to control the topology of consumers and producers that are services, as well as external entities. For example, a video camera controlled over an IEEE 1394B bus can be registered as a Producer service in the Framework's service registry and a TV, also connected to this bus, can be registered as a Consumer service. It would be very inefficient to stream the video data through the OSGi environment. Therefore, the Wire Admin service can be used to supply the external addressing information to the camera and the monitor to make a direct connection *outside* the OSGi environment. The Wire Admin service provides a uniform mechanism to connect both external entities and internal entities.

Figure 108.11 Connecting External Entities



A Consumer service and a Producer service associated with a Wire object receive enough information to establish a direct link because the PIDs of both services are in the Wire object's properties.

This situation, however, does not guarantee *compatibility* between Producer and the Consumer service. It is therefore recommended that flavors are used to ensure this compatibility. Producer services that participate in an external addressing scheme, like IEEE 1394B, should have a flavor that reflects this address. In this case, there should then for example be a IEEE 1394B address class. Consumer services that participate in this external addressing scheme should only accept data of this flavor.

The OSGi ???, defines the concept of a device category. This is a description of what classes and properties are used in a specific device category: for example, a UPnP device category that defines the interface that must be used to register for a UPnP device, among other things.

Device category descriptions should include a section that addresses the external wiring issue. This section should include what objects are send over the wire to exchange addressing information.

108.13 Related Standards

108.13.1 Java Beans

The Wire Admin service leverages the component architecture that the Framework service registry offers. Java Beans attempt to achieve similar goals. Java Beans are classes that follow a number of recommendations that allow them to be configured at run time. The techniques that are used by Java Beans during configuration are serialization and the construction of adapter classes.

Creating adapter classes in a resource constrained OSGi Framework was considered too heavy weight. Also, the dynamic nature of the OSGi environment, where services are registered and unregistered continuously, creates a mismatch between the intended target area of Java Beans and the OSGi Framework.

Also, Java Beans can freely communicate once they have a reference to each other. This freedom makes it impossible to control the communication between Java Beans.

This Wire Admin service specification was developed because it is lightweight and leverages the unique characteristics of the OSGi Framework. The concept of a Wire object that acts as an intermediate between the Producer and Consumer service allows the implementation of a security policy because both parties cannot communicate directly.

108.14 Security

108.14.1 Separation of Consumer and Producer Services

The Consumer and Producer service never directly communicate with each other. All communication takes place through a Wire object. This allows a Wire Admin service implementation to control the security aspects of creating a connection, and implies that the Wire Admin service must be a trusted service in a secure environment. Only one bundle should have the ServicePermission[WireAdmin, REGISTER].

ServicePermission[Producer|Consumer, REGISTER] should not be restricted. ServicePermission[Producer|Consumer, GET] must be limited to trusted bundles (the Wire Admin service implementation) because a bundle with this permission can call such services and access information that it should not be able to access.

108.14.2 Using Wire Admin Service

This specification assumes that only a few applications require access to the Wire Admin service. The WireAdmin interface contains all the security sensitive methods that create, update, and remove Wire objects. (This is the reason that the update and delete methods are on the WireAdmin interface

and not on the Wire interface). ServicePermission[WireAdmin,GET] should therefore only be given to trusted bundles that can manage the topology.

108.14.3 Wire Permission

Composite Producer and Consumer services can be restricted in their use of scope names. This restriction is managed with the WirePermission class. A WirePermission consists of a scope name and the action CONSUME or PRODUCE. The name used with the WirePermission may contain wild-cards as specified in the java.security.BasicPermission class.

108.15 org.osgi.service.wireadmin

Wire Admin Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.wireadmin; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.wireadmin; version="[1.0,1.1)"}
```

108.15.1 Summary

- BasicEnvelope - {@code BasicEnvelope} is an implementation of the {@link Envelope} interface
- Consumer - Data Consumer, a service that can receive updated values from {@link Producer} services.
- Envelope - Identifies a contained value.
- Producer - Data Producer, a service that can generate values to be used by {@link Consumer} services.
- Wire - A connection between a Producer service and a Consumer service.
- WireAdmin - Wire Administration service.
- WireAdminEvent - A Wire Admin Event.
- WireAdminListener - Listener for Wire Admin Events.
- WireConstants - Defines standard names for {@code Wire} properties, wire filter attributes, Consumer and Producer service properties.
- WirePermission - Permission for the scope of a {@code Wire} object.

108.15.2 public class BasicEnvelope implements Envelope

{@code BasicEnvelope} is an implementation of the {@link Envelope} interface

Concurrency Immutable

108.15.2.1 public BasicEnvelope(Object value, Object identification, String scope)

value Content of this envelope, may be {@code null}.

identification Identifying object for this {@code Envelope} object, must not be {@code null}

scope Scope name for this object, must not be {@code null}

- Constructor.

See Also `Envelope`

108.15.2.2 **public Object getIdentification()**

- Return the identification of this `{@code Envelope}` object. An identification may be of any Java type. The type must be mutually agreed between the Consumer and Producer services.

Returns an object which identifies the status item in the address space of the composite producer, must not be null.

See Also `org.osgi.service.wireadmin.Envelope.getIdentification()`

108.15.2.3 **public String getScope()**

- Return the scope name of this `{@code Envelope}` object. Scope names are used to restrict the communication between the Producer and Consumer services. Only `{@code Envelopes}` objects with a scope name that is permitted for the Producer and the Consumer services must be passed through a `{@code Wire}` object.

Returns the security scope for the status item, must not be null.

See Also `org.osgi.service.wireadmin.Envelope.getScope()`

108.15.2.4 **public Object getValue()**

- Return the value associated with this `{@code Envelope}` object.

Returns the value of the status item, or `{@code null}` when no item is associated with this object.

See Also `org.osgi.service.wireadmin.Envelope.getValue()`

108.15.3 **public abstract Consumer**

Data Consumer, a service that can receive updated values from `{@link Producer}` services.

Service objects registered under the `{@code Consumer}` interface are expected to consume values from a Producer service via a `{@code Wire}` object. A Consumer service may poll the Producer service by calling the `{@link Wire#poll()}` method. The Consumer service will also receive an updated value when called at it's `{@link #updated(Wire, Object)}` method. The Producer service should have coerced the value to be an instance of one of the types specified by the `{@link Wire#getFlavors()}` method, or one of their subclasses.

Consumer service objects must register with a `{@code service.pid}` and a `{@link WireConstants#WIREADMIN_CONSUMER_FLAVORS}` property. It is recommended that Consumer service objects also register with a `{@code service.description}` property.

If an `{@code Exception}` is thrown by any of the `{@code Consumer}` methods, a `{@code WireAdminEvent}` of type `{@link WireAdminEvent#CONSUMER_EXCEPTION}` is broadcast by the Wire Admin service.

Security Considerations - Data consuming bundles will require `{@code ServicePermission[Consumer,REGISTER]}`. In general, only the Wire Admin service bundle should have this permission. Thus only the Wire Admin service may directly call a Consumer service. Care must be taken in the sharing of `{@code Wire}` objects with other bundles.

Consumer services must be registered with their scope when they can receive different types of objects from the Producer service. The Consumer service should have `{@code WirePermission}` for each of these scope names.

108.15.3.1 **public abstract void producersConnected(Wire[][] wires)**

wires An array of the current and complete list of `{@code Wire}` objects to which this Consumer service is connected. May be `{@code null}` if the Consumer service is not currently connected to any `{@code Wire}` objects.

- Update the list of {@code Wire} objects to which this Consumer service is connected.

This method is called when the Consumer service is first registered and subsequently whenever a {@code Wire} associated with this Consumer service becomes connected, is modified or becomes disconnected.

The Wire Admin service must call this method asynchronously. This implies that implementors of Consumer can be assured that the callback will not take place during registration when they execute the registration in a synchronized method.

108.15.3.2 **public abstract void updated(Wire wire, Object value)**

wire The {@code Wire} object which is delivering the updated value.

value The updated value. The value should be an instance of one of the types specified by the {@link Wire#getFlavors()} method.

- Update the value. This Consumer service is called by the {@code Wire} object with an updated value from the Producer service.

Note: This method may be called by a {@code Wire} object prior to this object being notified that it is connected to that {@code Wire} object (via the {@link #producersConnected(Wire[])} method).

When the Consumer service can receive {@code Envelope} objects, it must have registered all scope names together with the service object, and each of those names must be permitted by the bundle's {@code WirePermission}. If an {@code Envelope} object is delivered with the {@code updated} method, then the Consumer service should assume that the security check has been performed.

108.15.4 **public abstract Envelope**

Identifies a contained value. An {@code Envelope} object combines a status value, an identification object and a scope name. The {@code Envelope} object allows the use of standard Java types when a Producer service can produce more than one kind of object. The {@code Envelope} object allows the Consumer service to recognize the kind of object that is received. For example, a door lock could be represented by a {@code Boolean} object. If the {@code Producer} service would send such a {@code Boolean} object, then the Consumer service would not know what door the {@code Boolean} object represented. The {@code Envelope} object contains an identification object so the Consumer service can discriminate between different kinds of values. The identification object may be a simple {@code String} object, but it can also be a domain specific object that is mutually agreed by the Producer and the Consumer service. This object can then contain relevant information that makes the identification easier.

The scope name of the envelope is used for security. The Wire object must verify that any {@code Envelope} object send through the {@code update} method or coming from the {@code poll} method has a scope name that matches the permissions of both the Producer service and the Consumer service involved. The wireadmin package also contains a class {@code BasicEnvelope} that implements the methods of this interface.

See Also `WirePermission`, `BasicEnvelope`

108.15.4.1 **public abstract Object getIdentification()**

- Return the identification of this {@code Envelope} object. An identification may be of any Java type. The type must be mutually agreed between the Consumer and Producer services.

Returns an object which identifies the status item in the address space of the composite producer, must not be null.

108.15.4.2 **public abstract String getScope()**

- Return the scope name of this {@code Envelope} object. Scope names are used to restrict the communication between the Producer and Consumer services. Only {@code Envelopes} objects with a

scope name that is permitted for the Producer and the Consumer services must be passed through a {@code Wire} object.

Returns the security scope for the status item, must not be null.

108.15.4.3 **public abstract Object getValue()**

- Return the value associated with this {@code Envelope} object.

Returns the value of the status item, or {@code null} when no item is associated with this object.

108.15.5 **public abstract Producer**

Data Producer, a service that can generate values to be used by {@link Consumer} services.

Service objects registered under the Producer interface are expected to produce values (internally generated or from external sensors). The value can be of different types. When delivering a value to a {@code Wire} object, the Producer service should coerce the value to be an instance of one of the types specified by {@link Wire#getFlavors()}. The classes are specified in order of preference.

When the data represented by the Producer object changes, this object should send the updated value by calling the {@code update} method on each of {@code Wire} objects passed in the most recent call to this object's {@link #consumersConnected(Wire[])} method. These {@code Wire} objects will pass the value on to the associated {@code Consumer} service object.

The Producer service may use the information in the {@code Wire} object's properties to schedule the delivery of values to the {@code Wire} object.

Producer service objects must register with a {@code service.pid} and a {@link WireConstants#WIREADMIN_PRODUCER_FLAVORS} property. It is recommended that a Producer service object also registers with a {@code service.description} property. Producer service objects must register with a {@link WireConstants#WIREADMIN_PRODUCER_FILTERS} property if the Producer service will be performing filtering instead of the {@code Wire} object.

If an exception is thrown by a Producer object method, a {@code WireAdminEvent} of type {@link WireAdminEvent#PRODUCER_EXCEPTION} is broadcast by the Wire Admin service.

Security Considerations. Data producing bundles will require {@code ServicePermission[Producer,REGISTER]} to register a Producer service. In general, only the Wire Admin service should have {@code ServicePermission[Producer,GET]}. Thus only the Wire Admin service may directly call a Producer service. Care must be taken in the sharing of {@code Wire} objects with other bundles.

Producer services must be registered with scope names when they can send different types of objects (composite) to the Consumer service. The Producer service should have {@code WirePermission} for each of these scope names.

108.15.5.1 **public abstract void consumersConnected(Wire[][] wires)**

wires An array of the current and complete list of {@code Wire} objects to which this Producer service is connected. May be {@code null} if the Producer is not currently connected to any {@code Wire} objects.

- Update the list of {@code Wire} objects to which this {@code Producer} object is connected.

This method is called when the Producer service is first registered and subsequently whenever a {@code Wire} associated with this Producer becomes connected, is modified or becomes disconnected.

The Wire Admin service must call this method asynchronously. This implies that implementors of a Producer service can be assured that the callback will not take place during registration when they execute the registration in a synchronized method.

108.15.5.2 public abstract Object polled(Wire wire)

wire The {@code Wire} object which is polling this service.

- Return the current value of this {@code Producer} object.

This method is called by a {@code Wire} object in response to the Consumer service calling the {@code Wire} object's {@code poll} method. The Producer should coerce the value to be an instance of one of the types specified by {@code Wire#getFlavors()}. The types are specified in order of preference. The returned value should be as new or newer than the last value furnished by this object.

Note: This method may be called by a {@code Wire} object prior to this object being notified that it is connected to that {@code Wire} object (via the {@code #consumersConnected(Wire[])} method).

If the Producer service returns an {@code Envelope} object that has an impermissible scope name, then the Wire object must ignore (or remove) the transfer.

If the {@code Wire} object has a scope set, the return value must be an array of {@code Envelope} objects ({@code Envelope[]}). The {@code Wire} object must have removed any {@code Envelope} objects that have a scope name that is not in the Wire object's scope.

Returns The current value of the Producer service or {@code null} if the value cannot be coerced into a compatible type. Or an array of {@code Envelope} objects.

108.15.6 public abstract Wire

A connection between a Producer service and a Consumer service.

A {@code Wire} object connects a Producer service to a Consumer service. Both the Producer and Consumer services are identified by their unique {@code service.pid} values. The Producer and Consumer services may communicate with each other via {@code Wire} objects that connect them. The Producer service may send updated values to the Consumer service by calling the {@code #update(Object)} method. The Consumer service may request an updated value from the Producer service by calling the {@code #poll()} method.

A Producer service and a Consumer service may be connected through multiple {@code Wire} objects.

Security Considerations. {@code Wire} objects are available to Producer and Consumer services connected to a given {@code Wire} object and to bundles which can access the {@code WireAdmin} service. A bundle must have {@code ServicePermission[WireAdmin,GET]} to get the {@code WireAdmin} service to access all {@code Wire} objects. A bundle registering a Producer service or a Consumer service must have the appropriate {@code ServicePermission[Consumer!Producer,REGISTER]} to register the service and will be passed {@code Wire} objects when the service object's {@code consumersConnected} or {@code producersConnected} method is called.

Scope. Each Wire object can have a scope set with the {@code setScope} method. This method should be called by a Consumer service when it assumes a Producer service that is composite (supports multiple information items). The names in the scope must be verified by the {@code Wire} object before it is used in communication. The semantics of the names depend on the Producer service and must not be interpreted by the Wire Admin service.

No Implement Consumers of this API must not implement this interface

108.15.6.1 public abstract Class<?>[] getFlavors()

- Return the list of data types understood by the Consumer service connected to this {@code Wire} object. Note that subclasses of the classes in this list are acceptable data types as well.

The list is the value of the {@code WireConstants#WIREADMIN_CONSUMER_FLAVORS} service property of the Consumer service object connected to this object. If no such property was registered or the type of the property value is not {@code Class[]}, this method must return {@code null}.

Returns An array containing the list of classes understood by the Consumer service or `{@code null}` if the `{@code Wire}` is not connected, or the consumer did not register a `{@link WireConstants#WIREADMIN_CONSUMER_FLAVORS}` property or the value of the property is not of type `{@code Class[]}`.

108.15.6.2 **public abstract Object getLastValue()**

- Return the last value sent through this `{@code Wire}` object.

The returned value is the most recent, valid value passed to the `{@link #update(Object)}` method or returned by the `{@link #poll()}` method of this object. If filtering is performed by this `{@code Wire}` object, this methods returns the last value provided by the Producer service. This value may be an `{@code Envelope[]}` when the Producer service uses scoping. If the return value is an `Envelope` object (or array), it must be verified that the Consumer service has the proper `WirePermission` to see it.

Returns The last value passed though this `{@code Wire}` object or `{@code null}` if no valid values have been passed or the Consumer service has no permission.

108.15.6.3 **public abstract Dictionary<String, Object> getProperties()**

- Return the wire properties for this `{@code Wire}` object.

Returns The properties for this `{@code Wire}` object. The returned `{@code Dictionary}` must be read only.

108.15.6.4 **public abstract String[][] getScope()**

- Return the calculated scope of this `{@code Wire}` object. The purpose of the `{@code Wire}` object's scope is to allow a Producer and/or Consumer service to produce/consume different types over a single `{@code Wire}` object (this was deemed necessary for efficiency reasons). Both the Consumer service and the Producer service must set an array of scope names (their scope) with the service registration property `{@code WIREADMIN_PRODUCER_SCOPE}`, or `{@code WIREADMIN_CONSUMER_SCOPE}` when they can produce multiple types. If a Producer service can produce different types, it should set this property to the array of scope names it can produce, the Consumer service must set the array of scope names it can consume. The scope of a `{@code Wire}` object is defined as the intersection of permitted scope names of the Producer service and Consumer service.

If neither the Consumer, or the Producer service registers scope names with its service registration, then the `{@code Wire}` object's scope must be `{@code null}`.

The `{@code Wire}` object's scope must not change when a Producer or Consumer services modifies its scope.

A scope name is permitted for a Producer service when the registering bundle has `{@code WirePermission[name,PRODUCE]}`, and for a Consumer service when the registering bundle has `{@code WirePermission[name,CONSUME]}`.

If either Consumer service or Producer service has not set a `{@code WIREADMIN_*_SCOPE}` property, then the returned value must be `{@code null}`.

If the scope is set, the `{@code Wire}` object must enforce the scope names when `{@code Envelope}` objects are used as a parameter to update or returned from the `{@code poll}` method. The `{@code Wire}` object must then remove all `{@code Envelope}` objects with a scope name that is not permitted.

Returns A list of permitted scope names or null if the Produce or Consumer service has set no scope names.

108.15.6.5 **public abstract boolean hasScope(String name)**

name The scope name

- Return true if the given name is in this `{@code Wire}` object's scope.

Returns true if the name is listed in the permitted scope names

108.15.6.6 public abstract boolean isConnected()

- Return the connection state of this {@code Wire} object.

A {@code Wire} is connected after the Wire Admin service receives notification that the Producer service and the Consumer service for this {@code Wire} object are both registered. This method will return `true` prior to notifying the Producer and Consumer services via calls to their respective `consumersConnected` and `producersConnected` methods.

A {@code WireAdminEvent} of type `WireAdminEvent#WIRE_CONNECTED` must be broadcast by the Wire Admin service when the {@code Wire} becomes connected.

A {@code Wire} object is disconnected when either the Consumer or Producer service is unregistered or the {@code Wire} object is deleted.

A {@code WireAdminEvent} of type `WireAdminEvent#WIRE_DISCONNECTED` must be broadcast by the Wire Admin service when the {@code Wire} becomes disconnected.

Returns `true` if both the Producer and Consumer for this {@code Wire} object are connected to the {@code Wire} object; `false` otherwise.

108.15.6.7 public abstract boolean isValid()

- Return the state of this {@code Wire} object.

A connected {@code Wire} must always be disconnected before becoming invalid.

Returns `false` if this {@code Wire} object is invalid because it has been deleted via `WireAdmin#deleteWire(Wire)`; `true` otherwise.

108.15.6.8 public abstract Object poll()

- Poll for an updated value.

This method is normally called by the Consumer service to request an updated value from the Producer service connected to this {@code Wire} object. This {@code Wire} object will call the `Producer#poll(Wire)` method to obtain an updated value. If this {@code Wire} object is not connected, then the Producer service must not be called.

If this {@code Wire} object has a scope, then this method must return an array of `Envelope` objects. The objects returned must match the scope of this object. The {@code Wire} object must remove all `Envelope` objects with a scope name that is not in the {@code Wire} object's scope. Thus, the list of objects returned must only contain `Envelope` objects with a permitted scope name. If the array becomes empty, `null` must be returned.

A {@code WireAdminEvent} of type `WireAdminEvent#WIRE_TRACE` must be broadcast by the Wire Admin service after the Producer service has been successfully called.

Returns A value whose type should be one of the types returned by `getFlavors()`, `Envelope`, or `null` if the {@code Wire} object is not connected, the Producer service threw an exception, or the Producer service returned a value which is not an instance of one of the types returned by `getFlavors()`.

108.15.6.9 public abstract void update(Object value)

value The updated value. The value should be an instance of one of the types returned by `getFlavors()`.

- Update the value.

This method is called by the Producer service to notify the Consumer service connected to this {@code Wire} object of an updated value.

If the properties of this {@code Wire} object contain a `WireConstants#WIREADMIN_FILTER` property, then filtering is performed. If the Producer service connected to this {@code Wire} object was registered with the service property `WireConstants#WIREADMIN_PRODUC-`

ER_FILTERS}, the Producer service will perform the filtering according to the rules specified for the filter. Otherwise, this {@code Wire} object will perform the filtering of the value.

If no filtering is done, or the filter indicates the updated value should be delivered to the Consumer service, then this {@code Wire} object must call the {@code Consumer#updated(Wire, Object)} method with the updated value. If this {@code Wire} object is not connected, then the Consumer service must not be called and the value is ignored.

If the value is an {@code Envelope} object, and the scope name is not permitted, then the {@code Wire} object must ignore this call and not transfer the object to the Consumer service.

A {@code WireAdminEvent} of type {@code WireAdminEvent#WIRE_TRACE} must be broadcast by the Wire Admin service after the Consumer service has been successfully called.

See Also `WireConstants.WIREADMIN_FILTER`

108.15.7 public abstract WireAdmin

Wire Administration service.

This service can be used to create {@code Wire} objects connecting a Producer service and a Consumer service. {@code Wire} objects also have wire properties that may be specified when a {@code Wire} object is created. The Producer and Consumer services may use the {@code Wire} object's properties to manage or control their interaction. The use of {@code Wire} object's properties by a Producer or Consumer services is optional.

Security Considerations: A bundle must have {@code ServicePermission[WireAdmin,GET]} to get the Wire Admin service to create, modify, find, and delete {@code Wire} objects.

No Implement Consumers of this API must not implement this interface

108.15.7.1 public abstract Wire createWire(String producerPID, String consumerPID, Dictionary<String, ?> properties)

producerPID The {@code service.pid} of the Producer service to be connected to the {@code Wire} object.

consumerPID The {@code service.pid} of the Consumer service to be connected to the {@code Wire} object.

properties The {@code Wire} object's properties. This argument may be {@code null} if the caller does not wish to define any {@code Wire} object's properties.

- ❑ Create a new {@code Wire} object that connects a Producer service to a Consumer service. The Producer service and Consumer service do not have to be registered when the {@code Wire} object is created.

The {@code Wire} configuration data must be persistently stored. All {@code Wire} connections are reestablished when the {@code WireAdmin} service is registered. A {@code Wire} can be permanently removed by using the {@code #deleteWire(Wire)} method.

The {@code Wire} object's properties must have case insensitive {@code String} objects as keys (like the Framework). However, the case of the key must be preserved.

The {@code WireAdmin} service must automatically add the following {@code Wire} properties:

- {@code WireConstants#WIREADMIN_PID} set to the value of the {@code Wire} object's persistent identity (PID). This value is generated by the Wire Admin service when a {@code Wire} object is created.
- {@code WireConstants#WIREADMIN_PRODUCER_PID} set to the value of Producer service's PID.
- {@code WireConstants#WIREADMIN_CONSUMER_PID} set to the value of Consumer service's PID.

If the {@code properties} argument already contains any of these keys, then the supplied values are replaced with the values assigned by the Wire Admin service.

The Wire Admin service must broadcast a {@code WireAdminEvent} of type {@link WireAdminEvent#WIRE_CREATED} after the new {@code Wire} object becomes available from {@link #getWires(String)}.

Returns The {@code Wire} object for this connection.

Throws `IllegalArgumentException` – If {@code properties} contains invalid wire types or case variants of the same key name.

108.15.7.2 **public abstract void deleteWire(Wire wire)**

wire The {@code Wire} object which is to be deleted.

- Delete a {@code Wire} object.

The {@code Wire} object representing a connection between a Producer service and a Consumer service must be removed. The persistently stored configuration data for the {@code Wire} object must be destroyed. The {@code Wire} object's method {@link Wire#isValid()} will return {@code false} after it is deleted.

The Wire Admin service must broadcast a {@code WireAdminEvent} of type {@link WireAdminEvent#WIRE_DELETED} after the {@code Wire} object becomes invalid.

108.15.7.3 **public abstract Wire[][] getWires(String filter) throws InvalidSyntaxException**

filter Filter string to select {@code Wire} objects or {@code null} to select all {@code Wire} objects.

- Return the {@code Wire} objects that match the given {@code filter}.

The list of available {@code Wire} objects is matched against the specified {@code filter}. {@code Wire} objects which match the {@code filter} must be returned. These {@code Wire} objects are not necessarily connected. The Wire Admin service should not return invalid {@code Wire} objects, but it is possible that a {@code Wire} object is deleted after it was placed in the list.

The filter matches against the {@code Wire} object's properties including {@link WireConstants#WIREADMIN_PRODUCER_PID}, {@link WireConstants#WIREADMIN_CONSUMER_PID} and {@link WireConstants#WIREADMIN_PID}.

Returns An array of {@code Wire} objects which match the {@code filter} or {@code null} if no {@code Wire} objects match the {@code filter}.

Throws `InvalidSyntaxException` – If the specified {@code filter} has an invalid syntax.

See Also `org.osgi.framework.Filter`

108.15.7.4 **public abstract void updateWire(Wire wire, Dictionary<String, ?> properties)**

wire The {@code Wire} object which is to be updated.

properties The new {@code Wire} object's properties or {@code null} if no properties are required.

- Update the properties of a {@code Wire} object. The persistently stored configuration data for the {@code Wire} object is updated with the new properties and then the Consumer and Producer services will be called at the respective {@link Consumer#producersConnected(Wire[])} and {@link Producer#consumersConnected(Wire[])} methods.

The Wire Admin service must broadcast a {@code WireAdminEvent} of type {@link WireAdminEvent#WIRE_UPDATED} after the updated properties are available from the {@code Wire} object.

Throws `IllegalArgumentException` – If {@code properties} contains invalid wire types or case variants of the same key name.

108.15.8 **public class WireAdminEvent**

A Wire Admin Event.

{@code WireAdminEvent} objects are delivered to all registered {@code WireAdminListener} service objects which specify an interest in the {@code WireAdminEvent} type. Events must be delivered in chronological order with respect to each listener. For example, a {@code WireAdminEvent} of type {@link #WIRE_CONNECTED} must be delivered before a {@code WireAdminEvent} of type {@link #WIRE_DISCONNECTED} for a particular {@code Wire} object.

A type code is used to identify the type of event. The following event types are defined:

- {@link #WIRE_CREATED}
- {@link #WIRE_CONNECTED}
- {@link #WIRE_UPDATED}
- {@link #WIRE_TRACE}
- {@link #WIRE_DISCONNECTED}
- {@link #WIRE_DELETED}
- {@link #PRODUCER_EXCEPTION}
- {@link #CONSUMER_EXCEPTION}

Additional event types may be defined in the future.

Event type values must be unique and disjoint bit values. Event types must be defined as a bit in a 32 bit integer and can thus be bitwise ORed together.

Security Considerations. {@code WireAdminEvent} objects contain {@code Wire} objects. Care must be taken in the sharing of {@code Wire} objects with other bundles.

See Also `WireAdminListener`

Concurrency `Immutable`

108.15.8.1 **public static final int CONSUMER_EXCEPTION = 2**

A Consumer service method has thrown an exception.

This {@code WireAdminEvent} type indicates that a Consumer service method has thrown an exception. The {@link WireAdminEvent#getThrowable()} method will return the exception that the Consumer service method raised.

The value of {@code CONSUMER_EXCEPTION} is 0x00000002.

108.15.8.2 **public static final int PRODUCER_EXCEPTION = 1**

A Producer service method has thrown an exception.

This {@code WireAdminEvent} type indicates that a Producer service method has thrown an exception. The {@link WireAdminEvent#getThrowable()} method will return the exception that the Producer service method raised.

The value of {@code PRODUCER_EXCEPTION} is 0x00000001.

108.15.8.3 **public static final int WIRE_CONNECTED = 32**

The {@code WireAdminEvent} type that indicates that an existing {@code Wire} object has become connected. The Consumer object and the Producer object that are associated with the {@code Wire} object have both been registered and the {@code Wire} object is connected. See {@link Wire#isConnected()} for a description of the connected state. This event may come before the {@code producerConnected} and {@code consumersConnected} method have returned or called to allow synchronous delivery of the events. Both methods can cause other {@code WireAdminEvent}s to take place and requiring this event to be sent before these methods are returned would mandate asynchronous delivery.

The value of {@code WIRE_CONNECTED} is 0x00000020.

108.15.8.4 public static final int WIRE_CREATED = 4

A {@code Wire} has been created.

This {@code WireAdminEvent} type that indicates that a new {@code Wire} object has been created. An event is broadcast when {@link WireAdmin#createWire(String, String, java.util.Dictionary)} is called. The {@link WireAdminEvent#getWire()} method will return the {@code Wire} object that has just been created.

The value of {@code WIRE_CREATED} is 0x00000004.

108.15.8.5 public static final int WIRE_DELETED = 16

A {@code Wire} has been deleted.

This {@code WireAdminEvent} type that indicates that an existing wire has been deleted. An event is broadcast when {@link WireAdmin#deleteWire(Wire)} is called with a valid wire. {@link WireAdminEvent#getWire()} will return the {@code Wire} object that has just been deleted.

The value of {@code WIRE_DELETED} is 0x00000010.

108.15.8.6 public static final int WIRE_DISCONNECTED = 64

The {@code WireAdminEvent} type that indicates that an existing {@code Wire} object has become disconnected. The Consumer object or/and Producer object is/are unregistered breaking the connection between the two. See {@link Wire#isConnected} for a description of the connected state.

The value of {@code WIRE_DISCONNECTED} is 0x00000040.

108.15.8.7 public static final int WIRE_TRACE = 128

The {@code WireAdminEvent} type that indicates that a new value is transferred over the {@code Wire} object. This event is sent after the Consumer service has been notified by calling the {@link Consumer#updated(Wire, Object)} method or the Consumer service requested a new value with the {@link Wire#poll()} method. This is an advisory event meaning that when this event is received, another update may already have occurred and this the {@link Wire#getLastValue()} method returns a newer value then the value that was communicated for this event.

The value of {@code WIRE_TRACE} is 0x00000080.

108.15.8.8 public static final int WIRE_UPDATED = 8

A {@code Wire} has been updated.

This {@code WireAdminEvent} type that indicates that an existing {@code Wire} object has been updated with new properties. An event is broadcast when {@link WireAdmin#updateWire(Wire, java.util.Dictionary)} is called with a valid wire. The {@link WireAdminEvent#getWire()} method will return the {@code Wire} object that has just been updated.

The value of {@code WIRE_UPDATED} is 0x00000008.

108.15.8.9 public WireAdminEvent(ServiceReference<WireAdmin> reference, int type, Wire wire, Throwable exception)

reference The {@code ServiceReference} object of the Wire Admin service that created this event.

type The event type. See {@link #getType()}.

wire The {@code Wire} object associated with this event.

exception An exception associated with this event. This may be {@code null} if no exception is associated with this event.

- Constructs a {@code WireAdminEvent} object from the given {@code ServiceReference} object, event type, {@code Wire} object and exception.

108.15.8.10 public ServiceReference<WireAdmin> getServiceReference()

- Return the {@code ServiceReference} object of the Wire Admin service that created this event.

Returns The {@code ServiceReference} object for the Wire Admin service that created this event.

108.15.8.11 public Throwable getThrowable()

- Returns the exception associated with the event, if any.

Returns An exception or {@code null} if no exception is associated with this event.

108.15.8.12 public int getType()

- Return the type of this event.

The type values are:

- {@link #WIRE_CREATED}
- {@link #WIRE_CONNECTED}
- {@link #WIRE_UPDATED}
- {@link #WIRE_TRACE}
- {@link #WIRE_DISCONNECTED}
- {@link #WIRE_DELETED}
- {@link #PRODUCER_EXCEPTION}
- {@link #CONSUMER_EXCEPTION}

Returns The type of this event.

108.15.8.13 public Wire getWire()

- Return the {@code Wire} object associated with this event.

Returns The {@code Wire} object associated with this event or {@code null} when no {@code Wire} object is associated with the event.

108.15.9 public abstract WireAdminListener

Listener for Wire Admin Events.

{@code WireAdminListener} objects are registered with the Framework service registry and are notified with a {@code WireAdminEvent} object when an event is broadcast.

{@code WireAdminListener} objects can inspect the received {@code WireAdminEvent} object to determine its type, the {@code Wire} object with which it is associated, and the Wire Admin service that broadcasts the event.

{@code WireAdminListener} objects must be registered with a service property {@link WireConstants#WIREADMIN_EVENTS} whose value is a bitwise OR of all the event types the listener is interested in receiving.

For example:

```
Integer mask = Integer.valueOf(WIRE_TRACE | WIRE_CONNECTED | WIRE_DISCONNECTED);
Hashtable ht = new Hashtable();
ht.put(WIREADMIN_EVENTS, mask);
context.registerService(WireAdminListener.class.getName(), this, ht);
```

If a {@code WireAdminListener} object is registered without a service property {@link WireConstants#WIREADMIN_EVENTS}, then the {@code WireAdminListener} will receive no events.

Security Considerations. Bundles wishing to monitor {@code WireAdminEvent} objects will require {@code ServicePermission[WireAdminListener,REGISTER]} to register a {@code WireAdminListen-

er} service. Since {@code WireAdminEvent} objects contain {@code Wire} objects, care must be taken in assigning permission to register a {@code WireAdminListener} service.

See Also WireAdminEvent

108.15.9.1 **public abstract void wireAdminEvent(WireAdminEvent event)**

event The {@code WireAdminEvent} object.

- Receives notification of a broadcast {@code WireAdminEvent} object. The event object will be of an event type specified in this {@code WireAdminListener} service's {@link WireConstants#WIREADMIN_EVENTS} service property.

108.15.10 **public abstract WireConstants**

Defines standard names for {@code Wire} properties, wire filter attributes, Consumer and Producer service properties.

No Implement Consumers of this API must not implement this interface

108.15.10.1 **public static final String WIREADMIN_CONSUMER_COMPOSITE = "wireadmin.consumer.composite"**

A service registration property for a Consumer service that is composite. It contains the names of the composite Producer services it can cooperate with. Interoperability exists when any name in this array matches any name in the array set by the Producer service. The type of this property must be {@code String[]}.

108.15.10.2 **public static final String WIREADMIN_CONSUMER_FLAVORS = "wireadmin.consumer.flavors"**

Service Registration property (named {@code wireadmin.consumer.flavors}) specifying the list of data types understood by this Consumer service.

The Consumer service object must be registered with this service property. The list must be in the order of preference with the first type being the most preferred. The value of the property must be of type {@code Class[]}.

108.15.10.3 **public static final String WIREADMIN_CONSUMER_PID = "wireadmin.consumer.pid"**

{@code Wire} property key (named {@code wireadmin.consumer.pid}) specifying the {@code service.pid} of the associated Consumer service.

This wire property is automatically set by the Wire Admin service. The value of the property must be of type {@code String}.

108.15.10.4 **public static final String WIREADMIN_CONSUMER_SCOPE = "wireadmin.consumer.scope"**

Service registration property key (named {@code wireadmin.consumer.scope}) specifying a list of names that may be used to define the scope of this {@code Wire} object. A {@code Consumer} service should set this service property when it can produce more than one kind of value. This property is only used during registration, modifying the property must not have any effect of the {@code Wire} object's scope. Each name in the given list must have a {@code WirePermission[name, CONSUME]} or else is ignored. The type of this service registration property must be {@code String[]}.

See Also Wire.getScope(), WIREADMIN_PRODUCER_SCOPE

108.15.10.5 **public static final String WIREADMIN_EVENTS = "wireadmin.events"**

Service Registration property (named {@code wireadmin.events}) specifying the {@code WireAdminEvent} type of interest to a Wire Admin Listener service. The value of the property is a bitwise OR of all the {@code WireAdminEvent} types the Wire Admin Listener service wishes to receive and must be of type {@code Integer}.

See Also WireAdminEvent

108.15.10.6 `public static final String WIREADMIN_FILTER = "wireadmin.filter"`

{@code Wire} property key (named {@code wireadmin.filter}) specifying a filter used to control the delivery rate of data between the Producer and the Consumer service.

This property should contain a filter as described in the {@code Filter} class. The filter can be used to specify when an updated value from the Producer service should be delivered to the Consumer service. In many cases the Consumer service does not need to receive the data with the same rate that the Producer service can generate data. This property can be used to control the delivery rate.

The filter can use a number of predefined attributes that can be used to control the delivery of new data values. If the filter produces a match upon the wire filter attributes, the Consumer service should be notified of the updated data value.

If the Producer service was registered with the {@link #WIREADMIN_PRODUCER_FILTERS} service property indicating that the Producer service will perform the data filtering then the {@code Wire} object will not perform data filtering. Otherwise, the {@code Wire} object must perform basic filtering. Basic filtering includes supporting the following standard wire filter attributes:

- {@link #WIREVALUE_CURRENT} - Current value
- {@link #WIREVALUE_PREVIOUS} - Previous value
- {@link #WIREVALUE_DELTA_ABSOLUTE} - Absolute delta
- {@link #WIREVALUE_DELTA_RELATIVE} - Relative delta
- {@link #WIREVALUE_ELAPSED} - Elapsed time

See Also [org.osgi.framework.Filter](#)

108.15.10.7 `public static final String WIREADMIN_PID = "wireadmin.pid"`

{@code Wire} property key (named {@code wireadmin.pid}) specifying the persistent identity (PID) of this {@code Wire} object.

Each {@code Wire} object has a PID to allow unique and persistent identification of a specific {@code Wire} object. The PID must be generated by the {@link WireAdmin} service when the {@code Wire} object is created.

This wire property is automatically set by the Wire Admin service. The value of the property must be of type {@code String}.

108.15.10.8 `public static final String WIREADMIN_PRODUCER_COMPOSITE = "wireadmin.producer.composite"`

A service registration property for a Producer service that is composite. It contains the names of the composite Consumer services it can interoperate with. Interoperability exists when any name in this array matches any name in the array set by the Consumer service. The type of this property must be {@code String[]}.

108.15.10.9 `public static final String WIREADMIN_PRODUCER_FILTERS = "wireadmin.producer.filters"`

Service Registration property (named {@code wireadmin.producer.filters}). A {@code Producer} service registered with this property indicates to the Wire Admin service that the Producer service implements at least the filtering as described for the {@link #WIREADMIN_FILTER} property. If the Producer service is not registered with this property, the {@code Wire} object must perform the basic filtering as described in {@link #WIREADMIN_FILTER}.

The type of the property value is not relevant. Only its presence is relevant.

108.15.10.10 `public static final String WIREADMIN_PRODUCER_FLAVORS = "wireadmin.producer.flavors"`

Service Registration property (named {@code wireadmin.producer.flavors}) specifying the list of data types available from this Producer service.

The Producer service object should be registered with this service property.

The value of the property must be of type {@code Class[]}.

108.15.10.11

public static final String WIREADMIN_PRODUCER_PID = "wireadmin.producer.pid"

{@code Wire} property key (named {@code wireadmin.producer.pid}) specifying the {@code service.pid} of the associated Producer service.

This wire property is automatically set by the WireAdmin service. The value of the property must be of type {@code String}.

108.15.10.12

public static final String WIREADMIN_PRODUCER_SCOPE = "wireadmin.producer.scope"

Service registration property key (named {@code wireadmin.producer.scope}) specifying a list of names that may be used to define the scope of this {@code Wire} object. A Producer service should set this service property when it can produce more than one kind of value. This property is only used during registration, modifying the property must not have any effect of the {@code Wire} object's scope. Each name in the given list must have {@code WirePermission[name,PRODUCE]} or else is ignored. The type of this service registration property must be {@code String[]}.

See Also `Wire.getScope()`, `WIREADMIN_CONSUMER_SCOPE`

108.15.10.13

public static final String[] WIREADMIN_SCOPE_ALL

Matches all scope names.

108.15.10.14

public static final String WIREVALUE_CURRENT = "wirevalue.current"

{@code Wire} object's filter attribute (named {@code wirevalue.current}) representing the current value.

108.15.10.15

public static final String WIREVALUE_DELTA_ABSOLUTE = "wirevalue.delta.absolute"

{@code Wire} object's filter attribute (named {@code wirevalue.delta.absolute}) representing the absolute delta. The absolute (always positive) difference between the last update and the current value (only when numeric). This attribute must not be used when the values are not numeric.

108.15.10.16

public static final String WIREVALUE_DELTA_RELATIVE = "wirevalue.delta.relative"

{@code Wire} object's filter attribute (named {@code wirevalue.delta.relative}) representing the relative delta. The relative difference is $\frac{\text{@code previous} - \text{@code current}}{\text{@code current}}$ (only when numeric). This attribute must not be used when the values are not numeric.

108.15.10.17

public static final String WIREVALUE_ELAPSED = "wirevalue.elapsed"

{@code Wire} object's filter attribute (named {@code wirevalue.elapsed}) representing the elapsed time, in ms, between this filter evaluation and the last update of the {@code Consumer} service.

108.15.10.18

public static final String WIREVALUE_PREVIOUS = "wirevalue.previous"

{@code Wire} object's filter attribute (named {@code wirevalue.previous}) representing the previous value.

108.15.11

**public final class WirePermission
extends BasicPermission**

Permission for the scope of a {@code Wire} object. When a {@code Envelope} object is used for communication with the {@code poll} or {@code update} method, and the scope is set, then the {@code Wire} object must verify that the Consumer service has {@code WirePermission[name,CONSUME]} and the Producer service has {@code WirePermission[name,PRODUCE]} for all names in the scope.

The names are compared with the normal rules for permission names. This means that they may end with a "*" to indicate wildcards. E.g. `Door.*` indicates all scope names starting with the string "Door". The last period is required due to the implementations of the {@code BasicPermission} class.

Concurrency Thread-safe

108.15.11.1 **public static final String CONSUME = "consume"**

The action string for the {@code consume} action.

108.15.11.2 **public static final String PRODUCE = "produce"**

The action string for the {@code produce} action.

108.15.11.3 **public WirePermission(String name, String actions)**

name Wire name.

actions {@code produce}, {@code consume} (canonical order).

- Create a new WirePermission with the given name (may be wildcard) and actions.

108.15.11.4 **public boolean equals(Object obj)**

obj The object to test for equality.

- Determines the equality of two {@code WirePermission} objects. Checks that specified object has the same name and actions as this {@code WirePermission} object.

Returns true if {@code obj} is a {@code WirePermission}, and has the same name and actions as this {@code WirePermission} object; {@code false} otherwise.

108.15.11.5 **public String getActions()**

- Returns the canonical string representation of the actions. Always returns present actions in the following order: {@code produce}, {@code consume}.

Returns The canonical string representation of the actions.

108.15.11.6 **public int hashCode()**

- Returns the hash code value for this object.

Returns Hash code value for this object.

108.15.11.7 **public boolean implies(Permission p)**

p The permission to check against.

- Checks if this {@code WirePermission} object {@code implies} the specified permission.

More specifically, this method returns {@code true} if:

- *p* is an instance of the {@code WirePermission} class,
- *p*'s actions are a proper subset of this object's actions, and
- *p*'s name is implied by this object's name. For example, {@code java.*} implies {@code java.home}.

Returns {@code true} if the specified permission is implied by this object; {@code false} otherwise.

108.15.11.8 **public PermissionCollection newPermissionCollection()**

- Returns a new {@code PermissionCollection} object for storing {@code WirePermission} objects.

Returns A new {@code PermissionCollection} object suitable for storing {@code WirePermission} objects.

108.15.11.9 **public String toString()**

- Returns a string describing this {@code WirePermission}. The convention is to specify the class name, the permission name, and the actions in the following format: '(org.osgi.service.wireadmin.WirePermission "name" "actions")'.

Returns information about this {@code Permission} object.

108.16 References

- [1] *Design Patterns*
Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. Addison Wesley, ISBN 0-201-63361

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT